

Physics-Informed Neural Networks for 1D Heat Diffusion: Forward Solution and Diffusivity Identification from Noisy Data

Quentin Pontalier

Technical note — September 25, 2026

Abstract

This note summarizes a personal study of physics-informed neural networks (PINNs) on the one-dimensional heat equation. A compact JAX implementation with hard-constrained initial and boundary conditions and L-BFGS optimization solves the forward problem with a relative L^2 error of 6×10^{-5} without any data. The same network recovers the thermal diffusivity from about 25 noisy measurements within a few percent. The main difficulty of the inverse problem is *parameter identifiability*: only a restricted region of the space-time domain carries information about α . A two-stage sensitivity-weighted data loss is investigated as a way to exploit this structure. Code: https://github.com/qp-dev-ai/pinn-heat-diffusion_1D.

1 Problem

Following the PINN framework [1], we consider

$$\partial_t T = \alpha \partial_{xx} T, \quad x \in [0, 1], \quad t \in [0, 1.5], \quad T(x, 0) = \sin(\pi x), \quad T(0, t) = T(1, t) = 0, \quad (1)$$

with exact solution $T(x, t) = \sin(\pi x) e^{-\alpha \pi^2 t}$. The exact solution is used only to generate synthetic data and to evaluate errors. In the *forward* problem α is known. In the *inverse* problem α is unknown and must be estimated, together with T , from $N_d \approx 25$ measurements $T_i^{\text{obs}} = T(x_i, t_i) + \varepsilon_i$ at random locations. The noise is $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ with $\sigma = \eta \text{std}(T)$ and $\eta \in \{5, 10, 20\}\%$. Measurements with $T^{\text{obs}} < 0$ lie below the noise floor and are discarded.

2 Method

Network and hard constraints. A fully connected network $\mathcal{N}_\theta(x, t)$ (3 hidden layers of 16 neurons, tanh, Glorot-uniform initialization [2]) is embedded in the ansatz

$$T_\theta(x, t) = \sin(\pi x) + t x (1 - x) \mathcal{N}_\theta(x, t), \quad (2)$$

which satisfies the initial and boundary conditions exactly for any weights [3]. This removes the IC/BC penalty terms and their weighting coefficients.

Loss. The PDE residual $R = \partial_t T_\theta - \alpha \partial_{xx} T_\theta$ is computed by automatic differentiation on $N_f = 500$ collocation points drawn by Latin Hypercube Sampling. The loss is

$$\mathcal{L}(\theta, \alpha) = \frac{1}{N_f} \sum_{j=1}^{N_f} R(x_j, t_j)^2 + \lambda \frac{\sum_i w_i (T_\theta(x_i, t_i) - T_i^{\text{obs}})^2}{\sum_i w_i}, \quad (3)$$

with $w_i = 1$ unless stated otherwise. In the inverse problem, positivity of the diffusivity is enforced by the reparameterization $\alpha = \alpha_{\min} + \beta^2$, where β is trained jointly with θ .

Optimization. All computations use double precision. The loss is minimized with L-BFGS (jaxopt, zoom line search, 3000 iterations) from the initial weights, with no Adam warm-up. The collocation points are kept fixed during the optimization.

3 Forward problem

With $\alpha = 0.18$ known and no data, the PINN reaches a physics loss of 1.4×10^{-7} and a relative L^2 error of 6.3×10^{-5} on a 200×200 grid in about 30s on a laptop CPU (Fig. 1). The benchmark mainly validates the implementation. For a forward problem like this one, classical solvers remain faster and more accurate.

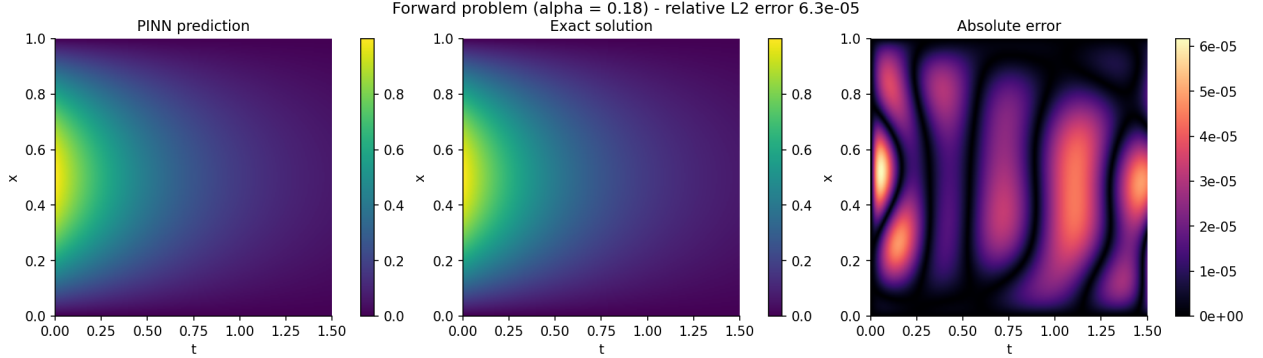


Figure 1: Forward problem: PINN prediction, exact solution and absolute error.

4 Inverse problem

Starting from $\alpha_0 = 1.0$, the PINN recovers $\hat{\alpha} = 0.1780$ (1.1% error) from 24 measurements with 10% noise, while reconstructing the full field (Fig. 2). The estimate first undershoots to $\alpha \approx 0.06$ during the early iterations. It then settles once the network has fitted the data.

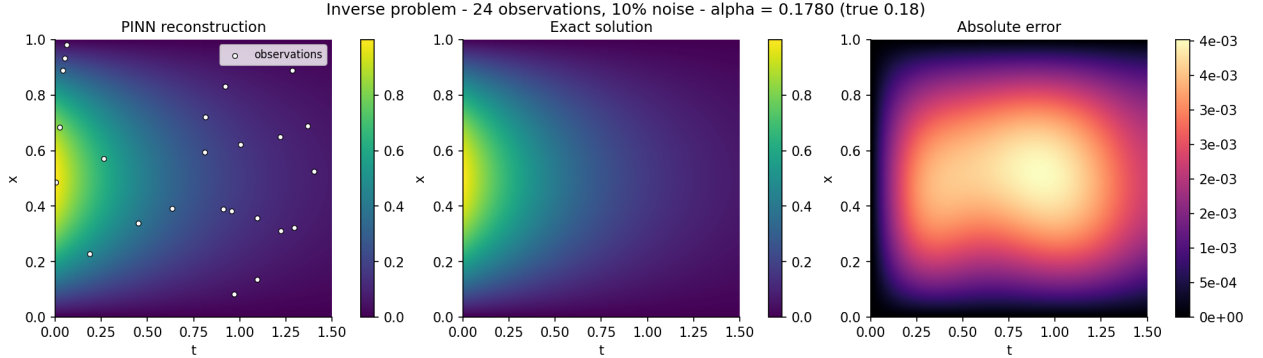


Figure 2: Inverse problem: reconstructed field with the measurement locations (white dots), exact solution and absolute error.

Identifiability. The information that a measurement carries about α is governed by the sensitivity [4]

$$S_\alpha(x, t) = \frac{\partial T}{\partial \alpha} = -\pi^2 t \sin(\pi x) e^{-\alpha \pi^2 t}. \quad (4)$$

It vanishes at $t = 0$, where the initial profile is independent of α , and decays at late times, when the signal falls below the noise. It is maximal at $t^* = 1/(\alpha \pi^2)$ and $x = 1/2$. For independent Gaussian errors, the contribution of a measurement to the Fisher information is proportional to S_α^2/σ^2 . Measurements that are accurate but insensitive therefore constrain α very little. For large α the informative window $t \sim t^*$ shrinks, and the temperature field can be fitted with a small error while α is still poorly determined.

Sensitivity-weighted data loss. To emphasize informative measurements, a two-stage procedure is used. First, an unweighted fit gives α_1 . Then the network is retrained with

$$w_i = \max\left(\left(|S_\alpha(x_i, t_i; \alpha_1)| / \max_k |S_\alpha(x_k, t_k; \alpha_1)|\right)^{1/2}, w_{\min}\right), \quad w_{\min} = 0.05. \quad (5)$$

The square root and the lower bound prevent the loss from concentrating on a few measurements. The weights are applied to the data term only, since the PDE residual must hold throughout the domain. In this benchmark S_α is evaluated analytically; in a real problem it would come from the sensitivity equation or from finite differences of the PINN.

5 Robustness study

Each configuration (α, η) was solved on three independent random datasets of 25 points, with and without weighting (Fig. 3, Table 1).

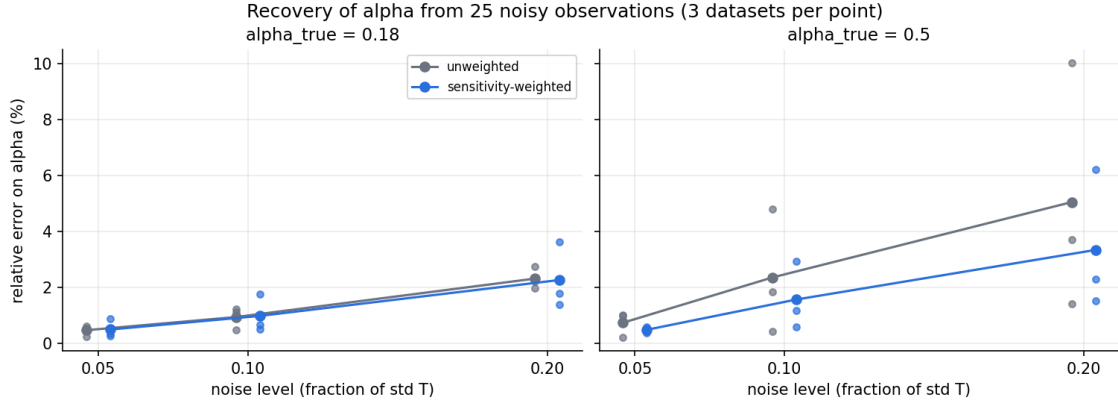


Figure 3: Relative error on $\hat{\alpha}$ against the noise level. Dots are individual datasets; lines are means.

α_{true}	noise	unweighted	sensitivity-weighted
0.18	5%	0.47 (0.62)	0.49 (0.88)
0.18	10%	0.94 (1.23)	0.98 (1.76)
0.18	20%	2.31 (2.76)	2.26 (3.63)
0.5	5%	0.73 (1.01)	0.48 (0.59)
0.5	10%	2.35 (4.81)	1.56 (2.93)
0.5	20%	5.05 (10.02)	3.34 (6.20)

Table 1: Relative error on $\hat{\alpha}$ (%): mean (max) over 3 datasets of 25 points, $\alpha_0 = 1.0$.

Discussion. Three observations stand out.

- **Accuracy.** α is recovered within 0.5% at 5% noise and within a few percent at 20% noise, starting from a guess that is 2 to 6 times too large. The error grows with the noise level and varies noticeably between datasets.
- **Effect of the weighting.** It depends on the regime. For $\alpha = 0.18$ it changes nothing measurable (mean errors within 0.05 percentage points). For $\alpha = 0.5$ it reduces the mean error by about one third at every noise level, and the worst case at 20% noise from 10.0% to 6.2%. This matches the sensitivity analysis: faster diffusion shrinks the window $t \sim t^* = 1/(\alpha\pi^2)$ in which measurements are informative, so down-weighting uninformative points matters more.
- **Systematic bias.** In all 36 cases α is *underestimated*. A likely cause is the removal of negative measurements. At late times the true temperature is close to zero, so the discarded points are preferentially those with negative noise. The retained late-time data are therefore biased upward, which mimics slower decay, i.e. a smaller α . Testing this would require repeating the study without the filtering.

6 Conclusions and perspectives

Hard constraints, float64 arithmetic and L-BFGS together give a PINN that is accurate and reproducible on this benchmark. The inverse problem is limited less by the network’s ability to approximate the field than

by how much information the data carry about the parameter. Every result should therefore be reported over several datasets and seeds. Adaptive loss balancing [5] is a complementary lever that was not used here. Next steps are:

- estimating the sensitivity without the analytical solution;
- weak-form (Galerkin) residual constraints;
- extension to nonlinear diffusion, 2D geometries and experimental data.

References

- [1] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [2] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feedforward neural networks, *AISTATS* (2010) 249–256.
- [3] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, *IEEE Trans. Neural Netw.* 9 (1998) 987–1000.
- [4] J. V. Beck, K. J. Arnold, *Parameter Estimation in Engineering and Science*, Wiley (1977).
- [5] S. Wang, Y. Teng, P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, *SIAM J. Sci. Comput.* 43 (2021) A3055–A3081.